

Security and Access Document

Planetic Web Automated Website, Domain, Hosting, DNS, Billing & Client Management Platform

1. Document Purpose

This document explains how security, access control, error handling, and edge-case protection should work in the Planetic Web platform.

The platform will manage sensitive customer services, including:

- Customer accounts
- Payments and invoices
- Domain registrations
- Hosting accounts
- Cloudflare DNS records
- Website project files
- Support tickets
- Renewal dates
- Admin actions

Because the platform connects with Stripe, Namecheap WHM, NameSilo or Namecheap domain APIs, Cloudflare, and cPanel email, security must be built from the beginning.

This document is written in plain English so both the founder and developer can understand what must be protected before launch.

2. Security Goals

The main security goals are:

1. Customers can only see and manage their own services.
 2. Admin users can manage the platform, but their actions must be logged.
 3. Staff users should only access what they need for their job.
 4. Payments must only be trusted after Stripe confirms them securely.
 5. External API keys must never be exposed publicly.
 6. Failed automation must not create duplicate domains, duplicate hosting accounts, or incorrect billing.
 7. Customer data must remain private.
 8. The platform must be safe from common web attacks such as password attacks, fake payment callbacks, form abuse, and unauthorized access.
-

3. Recommended Authentication Method

3.1 Authentication Method

The best authentication method for this app is:

```
Email + password login
Email verification
Password reset
Session-based authentication
Optional two-factor authentication for admins
```

3.2 Why This Is Best

This platform is not just a basic website. It manages billing, domains, hosting, DNS, and customer data.

Email and password login is the best starting point because:

- Customers understand it easily.
- It works well with Laravel.
- It allows account recovery.
- It allows email verification.
- It works well with customer dashboards.
- It avoids unnecessary complexity in the MVP.

3.3 Required Authentication Rules

The app must require:

- Customer registration before checkout or during checkout.
- Email verification before accessing sensitive dashboard features.
- Strong password requirements.
- Password reset by email.
- Login rate limiting to prevent brute-force attacks.
- Automatic logout after long inactivity.
- Admin users should use two-factor authentication where possible.

3.4 Password Rules

Passwords should require:

```
Minimum 10 characters
At least one uppercase letter
At least one lowercase letter
At least one number
At least one symbol
```

Customers should not be allowed to use very weak passwords such as:

```
password123  
123456789  
qwerty123  
planetic123
```

3.5 Session Rules

The app should:

- Use secure cookies.
- Use HTTPS only.
- Regenerate session after login.
- Log out inactive users after a reasonable time.
- Prevent one customer from using another customer's dashboard URL.

3.6 Two-Factor Authentication

Two-factor authentication should be:

```
Required for admins  
Optional for customers
```

For MVP, admin 2FA should be strongly recommended. If full 2FA is not ready for launch, admin accounts should at least use strong passwords and limited access.

4. User Roles

The platform should use role-based access control.

Recommended roles:

1. Guest
 2. Customer
 3. Support Staff
 4. Website Developer
 5. Billing Manager
 6. Technical Admin
 7. Super Admin
-

5. Role Permissions

5.1 Guest

A guest is someone who is not logged in.

Guest Can Do

- View public website pages.
- View pricing.
- Search for domain availability.
- View hosting packages.
- View the £200 website package.
- Contact the company.
- Register an account.
- Log in.

Guest Cannot Do

- Access customer dashboard.
 - Access admin dashboard.
 - View invoices.
 - Manage domains.
 - Manage hosting.
 - Open support tickets linked to an account.
 - Make changes to DNS.
 - View customer data.
 - Access payment history.
-

5.2 Customer

A customer is a logged-in user who has bought or is buying services.

Customer Can Do

- View their own dashboard.
- View their own orders.
- View their own invoices.
- View their own payments.
- View their own subscriptions.
- View their own domain names.
- View their own hosting accounts.
- View their own renewal dates.
- Complete website project intake forms.
- Upload website project files.
- Open support tickets.
- Reply to their own support tickets.
- Update their own billing/contact details.
- Update their payment method through Stripe.

- Request cancellation where allowed.

Customer Cannot Do

- View another customer's account.
 - View another customer's domains.
 - View another customer's invoices.
 - View another customer's support tickets.
 - Access admin dashboard.
 - Manually mark invoices as paid.
 - Manually change renewal dates.
 - Manually create hosting accounts.
 - Manually register domains without payment.
 - Access raw API responses.
 - View internal admin notes.
 - Change WHM package mapping.
 - Change Cloudflare API settings.
 - Change Stripe settings.
-

5.3 Support Staff

Support staff help customers with general issues.

Support Staff Can Do

- View customer profiles.
- View customer services.
- View support tickets.
- Reply to support tickets.
- Add internal ticket notes.
- View order status.
- View hosting/domain status.
- Resend basic customer emails.
- Escalate technical issues to Technical Admin.
- Escalate billing issues to Billing Manager.

Support Staff Cannot Do

- View full payment card details.
- Change payment status.
- Mark invoices as paid.
- Register domains manually.
- Create hosting accounts manually.
- Suspend or unsuspend hosting.
- Delete customer accounts.
- Change admin users.
- Change API keys.
- Change system settings.
- Access server credentials.
- Modify DNS records unless explicitly allowed later.

5.4 Website Developer

Website developer manages the manual website project work.

Website Developer Can Do

- View assigned website projects.
- View website intake form details.
- View uploaded project assets.
- Update website project status.
- Add project notes.
- Request more content from customer.
- Mark project as design in progress, review required, revisions in progress, launched, or completed.

Website Developer Cannot Do

- View full billing records.
- Change invoice status.
- Access payment settings.
- Access Stripe settings.
- Register domains.
- Create hosting accounts.
- Suspend hosting.
- Change DNS records unless needed and approved.
- Access unrelated customer support tickets.
- Manage admin users.
- Delete customer data.

5.5 Billing Manager

Billing manager handles payment, invoices, renewals, and failed payments.

Billing Manager Can Do

- View invoices.
- View payments.
- View subscriptions.
- View failed payments.
- View renewal dates.
- Resend invoices.
- Send payment reminders.
- Update billing notes.
- Cancel subscriptions where allowed.
- Escalate failed payments.
- Trigger hosting suspension request after grace period.
- View Stripe customer/payment reference IDs.

Billing Manager Cannot Do

- View full card numbers.
 - Directly edit Stripe secret keys.
 - Create hosting accounts.
 - Register domains manually.
 - Modify Cloudflare settings.
 - Change WHM settings.
 - Delete customers without Super Admin approval.
 - Change system-level configuration.
-

5.6 Technical Admin

Technical Admin manages hosting, domains, DNS, and provisioning failures.

Technical Admin Can Do

- View domains.
- View hosting accounts.
- View Cloudflare zones.
- View DNS records.
- Retry failed provisioning jobs.
- Create hosting account manually if automation fails.
- Suspend hosting.
- Unsuspend hosting.
- Review WHM errors.
- Review Cloudflare errors.
- Review registrar errors.
- Update technical service status.
- Sync domain/hosting status.
- Add technical notes.

Technical Admin Cannot Do

- Change Stripe secret keys.
 - Change business pricing without Super Admin approval.
 - Delete customer accounts.
 - Delete invoices.
 - Create new admin users.
 - Access customer passwords.
 - View raw payment card details.
 - Change company-wide security settings unless also Super Admin.
-

5.7 Super Admin

Super Admin is the owner or highest-level platform administrator.

Super Admin Can Do

- Manage all users.
- Manage all roles.
- Manage all customers.
- Manage all products and pricing.
- Manage all orders.
- Manage all payments and invoices.
- Manage all domains.
- Manage all hosting accounts.
- Manage all Cloudflare zones.
- Manage all website projects.
- Manage all support tickets.
- Retry or manually complete provisioning.
- Suspend or unsuspend hosting.
- Cancel services.
- View audit logs.
- Change system settings.
- Rotate API keys.
- Disable staff accounts.
- Create other admin users.

Super Admin Cannot Do

Even Super Admin should not be able to:

- View customer passwords.
- View full card numbers.
- Bypass Stripe payment verification for normal orders without audit logging.
- Delete audit logs.
- Expose API keys in frontend.
- Perform silent admin actions without logs.

6. Role Permission Summary

Feature	Customer	Support	Developer	Billing	Technical Admin	Super Admin
View own dashboard	Yes	No	No	No	No	Yes
View all customers	No	Yes	Limited	Yes	Yes	Yes
View invoices	Own only	Limited	No	Yes	Limited	Yes
Mark invoice paid	No	No	No	Limited/ Admin only	No	Yes
View domains	Own only	Yes	Limited	Limited	Yes	Yes

Feature	Customer	Support	Developer	Billing	Technical Admin	Super Admin
Register domains	Paid order only	No	No	No	Yes	Yes
Create hosting	Paid order only	No	No	No	Yes	Yes
Suspend hosting	No	No	No	Request only	Yes	Yes
Manage DNS	Limited future	No	Limited	No	Yes	Yes
View support tickets	Own only	Yes	Assigned only	Limited	Limited	Yes
Manage website projects	Own project view	Limited	Yes	No	Limited	Yes
Manage users/roles	No	No	No	No	No	Yes
Change API keys	No	No	No	No	No	Yes
View audit logs	No	No	No	Limited	Limited	Yes

7. Row-Level Security Rules

Row-level security means every database record must be protected so users only see records they are allowed to see.

Even if someone guesses a URL like:

```
/dashboard/invoices/55
```

they must not be able to see invoice 55 unless it belongs to them.

7.1 Main Rule

Every customer-owned table must include:

```
user_id
```

The app must always check that:

```
record.user_id = logged_in_user.id
```

before showing, editing, or deleting anything.

7.2 Customer Data Rules

Users Table

Rule

Customers can view and edit only their own profile.

Customer Can Access

```
users.id = their own user ID
```

Customer Cannot Access

```
any other user record  
admin user records  
staff user records
```

Orders Table

Rule

Customers can only see their own orders.

Access Rule

```
orders.user_id = logged_in_user.id
```

Customer Can

- View order details
- View order status
- View payment status

Customer Cannot

- Change order status
- Mark order paid
- Cancel completed order without proper workflow

Order Items Table

Rule

Customers can only see order items connected to their own orders.

Access Rule

```
order_items.order_id belongs to an order where orders.user_id =  
logged_in_user.id
```

Payments Table

Rule

Customers can only see their own payment records.

Access Rule

```
payments.user_id = logged_in_user.id
```

Important

Customers must never see:

- Stripe secret data
- Raw webhook payloads
- Internal failure logs
- Payment provider secret responses

Invoices Table

Rule

Customers can only see their own invoices.

Access Rule

```
invoices.user_id = logged_in_user.id
```

Customers can view/download invoices but cannot change invoice status.

Subscriptions Table

Rule

Customers can only see their own subscriptions.

Access Rule

```
subscriptions.user_id = logged_in_user.id
```

Customers can request cancellation or update payment method, but they cannot directly edit renewal dates or status.

Domains Table

Rule

Customers can only see their own domains.

Access Rule

```
domains.user_id = logged_in_user.id
```

Customers can view domain status and renewal date. DNS editing may be allowed later, but must be carefully controlled.

Domain Contacts Table

Rule

Customers can only see domain contacts connected to their own domains.

Access Rule

```
domain_contacts.domain_id belongs to a domain where domains.user_id =  
logged_in_user.id
```

Hosting Accounts Table

Rule

Customers can only see their own hosting accounts.

Access Rule

```
hosting_accounts.user_id = logged_in_user.id
```

Customers can view package, renewal date, status, and cPanel access link. They cannot manually change WHM package, server IP, or suspension status.

Cloudflare Zones Table

Rule

Customers can only see Cloudflare zone records linked to their own domains.

Access Rule

```
cloudflare_zones.user_id = logged_in_user.id
```

Customers should not see raw Cloudflare API tokens or internal API response logs.

DNS Records Table

Rule

Customers can only see DNS records connected to their own domains.

Access Rule

```
dns_records.user_id = logged_in_user.id
```

For MVP, customers may only view DNS records. Full DNS editing can be added later.

Website Projects Table

Rule

Customers can only see their own website projects.

Access Rule

```
website_projects.user_id = logged_in_user.id
```

Customers can submit intake details and upload files, but they cannot change internal project status directly.

Website Project Assets Table

Rule

Customers can only access files uploaded for their own projects.

Access Rule

```
website_project_assets.user_id = logged_in_user.id
```

Files must not be stored in a public folder without access checks.

Support Tickets Table

Rule

Customers can only see their own tickets.

Access Rule

```
support_tickets.user_id = logged_in_user.id
```

Customers cannot see internal notes.

Support Ticket Messages Table

Rule

Customers can only see messages from their own tickets, and only messages that are not internal notes.

Access Rule

```
support_ticket_messages.support_ticket_id belongs to a ticket where  
support_tickets.user_id = logged_in_user.id  
AND support_ticket_messages.is_internal_note = false
```

Provisioning Jobs Table

Rule

Customers should not directly access provisioning jobs.

Customers may see a simplified status such as:

```
Setting up your domain
Creating hosting account
DNS setup in progress
Completed
Manual review required
```

They should not see:

- API errors
- Raw request payloads
- Raw response payloads
- API keys
- Server details not intended for them

Audit Logs Table

Rule

Only Super Admin should access audit logs.

Customers and normal staff should not access audit logs.

8. Database Protection Rules

8.1 Never Trust URL IDs

The app must not trust IDs in URLs.

Bad example:

```
Show invoice where invoice_id = 20
```

Good example:

```
Show invoice where invoice_id = 20 AND user_id = logged_in_user.id
```

8.2 Every Customer Query Must Include Ownership Check

Any customer-facing database query must check the logged-in user.

8.3 Admin Actions Must Be Logged

When an admin changes anything important, log:

- Who performed the action
- What was changed
- Old value
- New value
- Time
- IP address
- User agent

8.4 Soft Delete Important Records

Do not permanently delete important records such as:

- Users
- Orders
- Payments
- Invoices
- Domains
- Hosting accounts
- Subscriptions

Use soft delete or status changes instead.

8.5 Protect Uploaded Files

Website project files must not be stored in a public folder where anyone can guess the file URL.

Files should be served through authenticated download routes.

9. Payment Security Rules

9.1 Never Trust Frontend Payment Success

The app must not create services just because the user returns to:

```
/checkout/success
```

That page can be visited manually.

Services must only be created after Stripe sends a verified webhook.

9.2 Stripe Webhook Must Be Verified

The Stripe webhook must verify that the request really came from Stripe.

If the webhook signature is invalid:

```
Reject the request  
Log the attempt  
Do not create services
```

9.3 Prevent Duplicate Webhook Processing

Stripe may send the same webhook more than once.

The app must store Stripe event IDs and ignore duplicates.

If the same event is received twice:

```
Do not create another order  
Do not register another domain  
Do not create another hosting account
```

9.4 No Full Card Storage

The app must never store full card numbers or CVV codes.

Stripe handles card storage. The app should only store Stripe customer IDs, payment intent IDs, subscription IDs, and invoice IDs.

10. API Key Security

The app will use several sensitive API keys:

```
Stripe secret key
Stripe webhook secret
WHM API token
Cloudflare API token
NameSilo API key
Namecheap API key
SMTP password
```

10.1 Storage Rule

API keys must be stored only in:

```
.env file
secure cPanel environment configuration
```

They must not be stored in:

```
public JavaScript
GitHub repository
database records visible to admins
frontend source code
logs
emails
support tickets
```

10.2 Access Rule

Only Super Admin or trusted developer should access API keys.

Support staff, billing staff, and website developers should not see API secrets.

10.3 Rotation Rule

API keys should be changed immediately if:

- A staff account is compromised
- A key is accidentally committed to GitHub
- A developer leaves
- Suspicious API activity is detected
- cPanel account is compromised

11. Error Handling Guide

This section explains what should happen when something fails.

11.1 Domain Search Fails

Possible Causes

- Registrar API is down.
- API key is wrong.
- Rate limit reached.
- Invalid domain format.
- Network timeout.

Customer Message

We could not check this domain right now. Please try again in a few moments.

Admin Action

- Log error.
- Show error in admin system health.
- Do not allow checkout for uncertain domain availability.
- Allow customer to retry.

11.2 Domain Is Shown Available but Registration Fails

Possible Causes

- Someone else registered it first.
- Registrar returned stale availability.
- Premium domain price changed.
- Contact details invalid.
- Payment succeeded but registrar rejected registration.

Customer Message

Your payment was successful, but the domain could not be registered automatically. Our team has been notified and will contact you with the next steps.

System Action

- Mark order as `manual_review`.
- Mark domain as `registration_failed`.
- Notify admin immediately.
- Do not silently ignore the failure.
- Do not create duplicate registration attempts without checks.
- Admin should offer alternative domain or refund domain portion if needed.

11.3 Payment Succeeds but Domain Registration Fails

System Action

- Keep payment record as successful.
- Mark order as manual review.
- Do not mark full order as completed.
- Notify admin.
- Show customer that order is being reviewed.
- Continue hosting setup only if appropriate.

Important

Do not refund automatically unless business rules say so. Some cases can be solved manually.

11.4 Payment Fails

Customer Message

Your payment could not be completed. Please check your card details or try another payment method.

System Action

- Keep order as unpaid.
 - Do not register domain.
 - Do not create hosting account.
 - Do not create Cloudflare zone.
 - Log failed payment reason.
 - Allow customer to retry checkout.
-

11.5 Stripe Webhook Fails

Possible Causes

- Invalid signature.
- Server error.
- Database error.
- Duplicate event.
- Timeout.

System Action

- Reject invalid signature.
- Store valid webhook event IDs.
- Ignore duplicates safely.
- Log error.

- Retry safe jobs.
 - Do not provision twice.
-

11.6 WHM Hosting Account Creation Fails

Possible Causes

- WHM API token invalid.
- Package name does not exist.
- Username already exists.
- Domain already exists in WHM.
- Account limit reached.
- Server quota reached.
- WHM server unavailable.

Customer Message

Your payment was successful. Your hosting account is being reviewed by our team and will be activated shortly.

System Action

- Mark hosting status as `failed` or `manual_review`.
 - Log WHM error.
 - Notify Technical Admin.
 - Do not retry blindly if duplicate account risk exists.
 - Allow Technical Admin to retry manually.
-

11.7 Cloudflare Zone Creation Fails

Possible Causes

- Cloudflare API token invalid.
- Zone already exists.
- Account permission issue.
- Cloudflare API unavailable.
- Domain invalid.

Customer Message

Your service has been created, but DNS setup is still in progress. Our team has been notified.

System Action

- Mark Cloudflare setup as failed.

- Notify Technical Admin.
 - Allow retry.
 - Do not block all order completion if hosting is already created, but show DNS pending.
-

11.8 Nameserver Update Fails

Possible Causes

- Registrar API error.
- Domain not fully registered yet.
- Nameserver format invalid.
- Registrar does not allow update immediately.
- API permission issue.

Customer Message

Your domain has been registered, but DNS activation is still in progress.

System Action

- Store Cloudflare nameservers.
 - Mark nameserver update as pending/failed.
 - Notify Technical Admin.
 - Retry later.
 - Show customer DNS activation pending.
-

11.9 DNS Record Creation Fails

Possible Causes

- Duplicate record.
- Invalid record value.
- Cloudflare permission issue.
- Zone not active yet.
- Server IP missing.

System Action

- Mark DNS records as failed.
 - Retry safe records.
 - Notify admin.
 - Do not duplicate existing records.
 - Keep email records DNS-only.
-

11.10 Email Sending Fails

Possible Causes

- SMTP password wrong.
- Mailbox not created.
- DNS records wrong.
- cPanel mail limit reached.
- Message blocked by spam filter.

Customer Impact

Customer may not receive order confirmation, renewal reminder, or support update.

System Action

- Log failed email.
 - Mark notification as failed.
 - Allow admin to resend email.
 - Show alert in admin dashboard.
 - Do not block order provisioning just because email failed.
-

11.11 Customer Upload Fails

Possible Causes

- File too large.
- Invalid file type.
- Storage permission error.
- Malware-like file detected.
- Upload interrupted.

Customer Message

Your file could not be uploaded. Please check the file type and size, then try again.

System Action

- Validate file type.
- Validate file size.
- Store securely.
- Do not allow executable files.
- Log repeated upload failures.

Allowed file types:

```
jpg
jpeg
png
webp
pdf
doc
docx
txt
zip only if needed
```

Do not allow:

```
php
exe
js
sh
bat
html
svg unless sanitized
```

11.12 Support Ticket Abuse

Possible Abuse

- Spam tickets.
- Very large messages.
- Malicious file uploads.
- Repeated ticket creation.

System Action

- Rate-limit ticket creation.
- Limit file size.
- Limit message length.
- Block dangerous file types.
- Allow admin to close abusive tickets.

11.13 Admin Login Attack

Possible Attack

Someone tries many passwords on admin login.

System Action

- Rate-limit login attempts.

- Lock account temporarily after repeated failures.
 - Notify Super Admin of suspicious admin login attempts.
 - Require strong password.
 - Use 2FA for admins where possible.
-

11.14 Customer Tries to Access Another Customer's Data

Example

Customer changes URL:

```
/dashboard/invoices/10
```

to:

```
/dashboard/invoices/11
```

System Action

- Check ownership before showing data.
- Return 404 or access denied.
- Log suspicious repeated attempts.

Customer Message

```
This record was not found.
```

Do not say:

```
You are trying to access another customer's invoice.
```

The safer response is to hide whether the record exists.

11.15 Duplicate Checkout

Possible Issue

Customer clicks pay twice or opens checkout in multiple tabs.

System Action

- Use unique cart/order references.
- Prevent duplicate active orders for the same checkout session.

- Do not register the same domain twice.
 - Do not create duplicate hosting accounts.
 - Mark duplicate attempts safely.
-

11.16 Domain Already Exists in WHM

Possible Issue

A domain may already exist on the hosting server.

System Action

- Check before creating hosting.
 - If domain exists, mark manual review.
 - Do not overwrite another account.
 - Notify Technical Admin.
-

11.17 Domain Already Exists in Cloudflare

Possible Issue

A Cloudflare zone may already exist.

System Action

- Check existing zones before creating new one.
 - Reuse zone if it belongs to your Cloudflare account and matches customer record.
 - Otherwise mark manual review.
-

12. Edge Cases to Handle Before Launch

12.1 Payment and Checkout Edge Cases

Handle these before launch:

- Customer pays but closes browser before success page.
- Stripe webhook arrives before local order is fully saved.
- Stripe sends same webhook twice.
- Payment succeeds but provisioning job fails.
- Payment fails but customer refreshes success URL.
- Customer starts checkout with one domain but tries to change it after payment.
- Coupon is applied but expires before checkout completes.
- Customer uses two checkout tabs at the same time.
- Customer pays for unavailable domain because availability changed.
- Stripe subscription is cancelled from Stripe dashboard but not synced locally.

- Customer disputes or charges back payment.
-

12.2 Domain Edge Cases

Handle:

- Domain becomes unavailable after search.
 - Premium domain price is different from normal price.
 - Registrar rejects contact details.
 - Invalid phone number format for domain contact.
 - Unsupported TLD.
 - TLD requires special fields.
 - Domain registration succeeds but local database update fails.
 - Domain expiry date is missing from API response.
 - Domain is registered but nameserver update fails.
 - Domain renewal payment fails.
 - Domain auto-renews at registrar before customer pays.
 - Customer wants to cancel domain after registration.
 - Customer enters a domain they already own.
 - Customer enters typo in domain name.
 - Domain transfer request is made before transfer feature exists.
-

12.3 Hosting Edge Cases

Handle:

- WHM package does not exist.
 - WHM username already exists.
 - Domain already exists in WHM.
 - WHM account limit reached.
 - Disk quota exceeded.
 - Server IP missing.
 - WHM API token expired.
 - WHM account created but database update fails.
 - Hosting created but DNS setup fails.
 - Customer payment fails on renewal but hosting remains active.
 - Customer pays overdue invoice and hosting needs unsuspension.
 - Hosting suspension fails.
 - Hosting unsuspension fails.
 - Customer tries to buy multiple hosting packages for same domain.
-

12.4 Cloudflare/DNS Edge Cases

Handle:

- Cloudflare zone already exists.

- Cloudflare token has wrong permissions.
 - Nameservers are assigned but not updated at registrar.
 - Registrar nameservers updated but Cloudflare zone remains pending.
 - DNS propagation takes longer than expected.
 - Duplicate DNS records.
 - MX records accidentally proxied.
 - Mail record accidentally proxied.
 - Customer changes DNS and breaks email.
 - DKIM record missing.
 - SPF record missing.
 - DMARC record missing.
 - Wrong server IP added to DNS.
 - SSL mode set incorrectly.
 - Cloudflare proxy breaks cPanel/webmail links.
-

12.5 Email Edge Cases

Handle:

- SMTP login fails.
 - Mailbox password changed.
 - cPanel mailbox does not exist.
 - Email goes to spam.
 - Renewal reminder not sent.
 - Failed payment email not sent.
 - Customer email address invalid.
 - Customer typo in email during registration.
 - Support reply email fails.
 - Email sending limit reached.
 - DKIM/SPF/DMARC missing or incorrect.
-

12.6 Account and Access Edge Cases

Handle:

- Customer tries to access another customer's invoice.
 - Customer tries to access another customer's support ticket.
 - Customer changes hidden form field user ID.
 - Staff tries to access Super Admin settings.
 - Deleted/suspended user tries to log in.
 - Unverified email tries to access dashboard.
 - Staff account is compromised.
 - Admin forgets password.
 - Admin leaves company and account remains active.
 - Customer requests account deletion while services are still active.
-

12.7 Website Project Edge Cases

Handle:

- Customer pays but does not submit website content.
 - Customer uploads harmful file.
 - Customer uploads very large files.
 - Customer uploads wrong logo format.
 - Customer requests too many pages beyond package scope.
 - Customer wants refund after work starts.
 - Customer approves design then requests major changes.
 - Customer domain/hosting is ready but website content is missing.
 - Admin marks project completed by mistake.
-

12.8 Support Edge Cases

Handle:

- Customer opens many duplicate tickets.
 - Customer sends abusive content.
 - Customer uploads malicious files.
 - Staff replies internally but note is accidentally visible.
 - Ticket assigned to inactive staff user.
 - Customer replies to closed ticket.
 - Customer asks for domain cancellation after registration.
 - Customer asks for data deletion.
-

13. Launch Security Checklist

Before launch, confirm:

```
HTTPS is active on planeticweb.com
APP_DEBUG=false
.env file is not public
No API keys are in GitHub
Stripe webhook signature verification is enabled
Admin passwords are strong
Admin 2FA is enabled or planned
Login rate limiting is enabled
Domain search rate limiting is enabled
Customer ownership checks are implemented
Customers cannot access other customers' records
Support internal notes are hidden from customers
File uploads are restricted
Dangerous file types are blocked
Stripe test keys are replaced with live keys
```

```
WHM API token is active and restricted where possible
Cloudflare API token has limited permissions
Registrar API key is stored securely
SMTP mailbox exists
SPF/DKIM/DMARC records are configured
Queue cron is running
Scheduler cron is running
Provisioning failures are logged
Admin can retry failed provisioning
Audit logs are enabled
Database backups are enabled
```

14. Security Rules for Developers

Developers must follow these rules:

1. Never expose API keys in frontend code.
2. Never trust payment success from frontend redirect.
3. Always verify Stripe webhooks.
4. Always check `user_id` before showing customer records.
5. Never allow customers to change hidden fields like price, user ID, order ID, or service status.
6. Always calculate prices on the server.
7. Always log important admin actions.
8. Never store customer card numbers.
9. Never store customer passwords in plain text.
10. Never store uploaded files in public access folders without permission checks.
11. Always validate file uploads.
12. Always validate domain names before sending to registrar.
13. Always make provisioning jobs safe to retry.
14. Always prevent duplicate domain registration.
15. Always prevent duplicate WHM account creation.
16. Always show safe error messages to customers.
17. Always log detailed errors privately for admin/developer review.
18. Never show raw API errors to customers.
19. Never allow staff to access more than their role requires.
20. Never delete audit logs.

15. Customer-Facing Error Message Rules

Customer messages should be simple and not reveal technical details.

Bad Message

```
WHM API returned 401 invalid token while calling createacct.
```

Good Message

Your hosting setup could not be completed automatically. Our team has been notified and will review it shortly.

Bad Message

Cloudflare API token permission error.

Good Message

DNS setup is still in progress. Our team has been notified.

Bad Message

Stripe webhook signature verification failed.

Good Message

We could not confirm the payment yet. Please contact support if payment was deducted.

16. Admin Error Message Rules

Admin users should see more detail, but still not exposed secrets.

Admin can see:

- Error type
- Failed integration
- Time of failure
- Order ID
- User ID
- External reference ID
- Safe API response message
- Retry button

Admin should not see:

- Full API keys
- SMTP password

- Stripe secret key
 - Customer password
 - Full card number
 - CVV
 - Unmasked sensitive tokens
-

17. Data Privacy Rules

The platform should collect only the data needed to provide service.

Customer data collected may include:

- Name
- Email
- Phone
- Company name
- Billing address
- Domain contact details
- Website project details
- Uploaded project files

The platform should not collect unnecessary sensitive data.

Customer Data Deletion

If a customer asks for deletion:

- Check if they have active services.
 - Cancel services first if required.
 - Keep legally required invoices/payment records.
 - Remove or anonymize personal data where possible.
 - Do not delete audit records needed for security and legal protection.
-

18. Backup and Recovery Security

The app should have:

- Daily database backups
- Backup of uploaded website project files
- Backup retention policy
- Secure backup storage
- Restricted backup access

Backups must not be stored in a public folder.

Backups must not be downloadable without authentication.

19. Final Security Recommendation

For MVP, the most important security protections are:

```
Verified Stripe webhooks
Strong customer ownership checks
Strict admin role permissions
Secure API key storage
Safe file uploads
Rate-limited login and domain search
Detailed private error logs
Simple customer-facing error messages
Retryable provisioning jobs
Admin audit logs
```

If these are implemented properly, the app will be secure enough for early launch while still being simple enough to build.

The platform should be designed with one main rule:

```
Customers can only access what belongs to them, staff can only access what
their job requires, and automation should only run after verified payment.
```